Introduction to PyTorch

Tensors, Neural Networks & Backpropagation

Dimitrios Dekas

What is PyTorch?

The official line

An open-source machine-learning framework that accelerates the path from research prototyping to production deployment, using an optimized tensor library for deep learning that utilizes both GPUs and CPUs.

What is PyTorch?

The official line

An open-source machine-learning framework that accelerates the path from research prototyping to production deployment, using an optimized tensor library for deep learning that utilizes both GPUs and CPUs.

In plain terms

NumPy that runs on a **GPU** and does your **calculus** for you, **enhanced** with useful neural-network building blocks.

PyTorch Trivia

Who is using it

Used in research and production by **Meta**, **Microsoft**, **OpenAI** and **Tesla**.

Why they use it

The **most-used** deep-learning framework in published research (Papers With Code). New ideas tend to land in PyTorch first.

Models you've heard of

Used for the models behind **ChatGPT**, **Llama**, **Stable Diffusion** and **Whisper**.

Where can you run PyTorch?

On your own machine

Pick your OS and compute backend (**CPU**, **CUDA** or **MPS**).

Easier when using the installation selector at **pytorch.org**.

In your browser

Google Colab and **Kaggle Notebooks** ship with PyTorch and a **free GPU**.

At scale

Cloud GPU VMs (AWS, GCP, Azure), **HPC clusters**, and **Docker** containers.

Your first lines of PyTorch

Everything starts with one import:

```
import torch
```

Then check which version you are running:

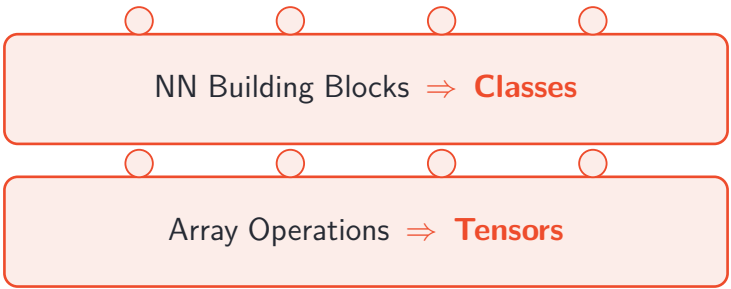
```
print(torch.__version__)        # e.g. 2.3.0+cu121
```

Set a seed so results are reproducible:

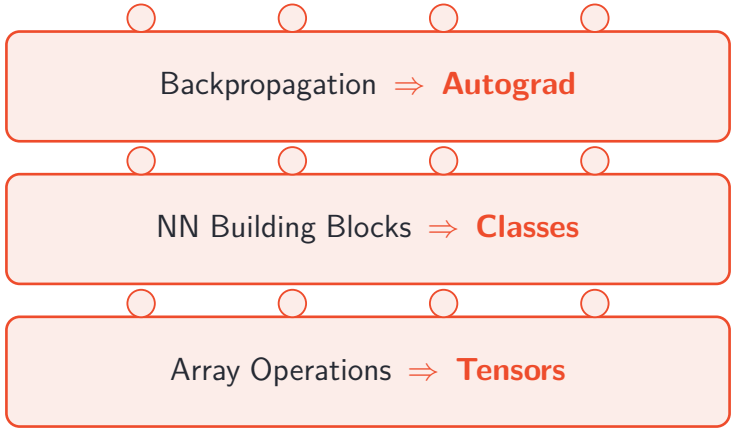
```
torch.manual_seed(42)
```

PyTorch Lego Pieces

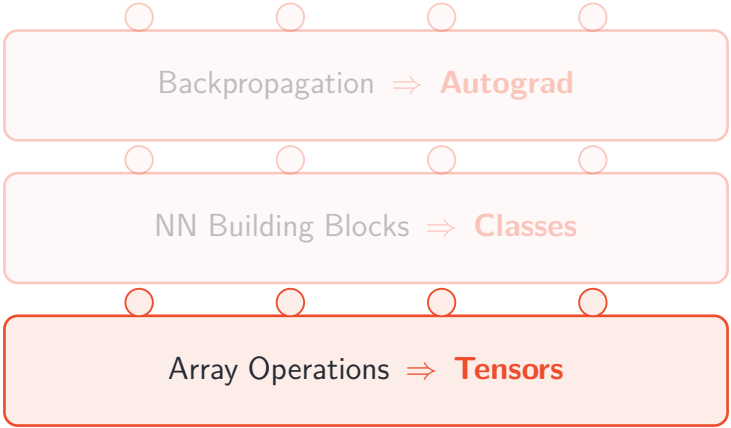
PyTorch Lego Pieces



PyTorch Lego Pieces



PyTorch Lego Pieces



Tensors

the data container that knows its own gradient

What is a Tensor?

A **Tensor** is a multi-dimensional matrix containing elements of a single data type.

Common data types:

- `torch.float32` / `float64` — floating point (the default for most maths)
- `torch.float16` / `bfloat16` — half precision (faster, less memory)
- `torch.int64` / `int32` — integers (indices, class labels)
- `torch.bool` — boolean values (useful for masking)
- `torch.complex64` — complex numbers

```
print(x.dtype)
```

Why not just use NumPy Arrays?

A tensor is a NumPy array with **two extra powers**:

1. **It can live on a GPU.** Move it to the device and every operation runs there, in parallel over thousands of cores.

Is there a GPU? Device-agnostic code

PyTorch runs the *exact same* code on CPU or GPU.

```
import torch

print(torch.cuda.is_available())            # True with an NVIDIA GPU (CUDA)
print(torch.backends.mps.is_available())    # True on Apple Silicon (M1/M2/...)

# write once, run anywhere
device = "cuda" if torch.cuda.is_available() else "cpu"

x = x.to(device)                            # move onto the device
print(x.device)                             # cuda:0   (or cpu)
```

Why not just use NumPy Arrays?

A tensor is a NumPy array with **two extra powers**:

1. **It can live on a GPU.** Move it to the device and every operation runs there, in parallel over thousands of cores.
2. **It tracks the operations applied to it**, so the framework can compute gradients automatically.

Start small: scalars

In NumPy you already know:

```
scalar = np.array(7)
```

PyTorch works the same way:

```
scalar = torch.tensor(7)

print(scalar.ndim)
print(scalar.shape)
print(scalar.numel())
print(scalar.dtype)
print(scalar.item())

scalar = torch.tensor(7, dtype=torch.float32, device=device)
print(scalar.dtype)
print(scalar.device)
```

Start small: scalars

In NumPy you already know:

```
scalar = np.array(7)
```

PyTorch works the same way:

```
scalar = torch.tensor(7)

print(scalar.ndim)     # 0                 -- no axes
print(scalar.shape)    # torch.Size([])    -- empty shape
print(scalar.numel())  # 1                 -- number of elements
print(scalar.dtype)    # torch.int64
print(scalar.item())   # 7                 -- back to a Python number

scalar = torch.tensor(7, dtype=torch.float32, device=device)
print(scalar.dtype)    # torch.float32
print(scalar.device)   # cpu               -- on a CPU-only machine
```

Step up: vectors

In NumPy you already know:

```
vector = np.array([7, 7])
```

PyTorch works the same way:

```
vector = torch.tensor([7, 7])

print(vector.ndim)
print(vector.shape)
print(vector.numel())
print(vector.dtype)
print(vector.tolist())

vector = torch.tensor([7, 7], dtype=torch.float32, device=device)
print(vector.dtype)
print(vector.device)
```

Step up: vectors

In NumPy you already know:

```
vector = np.array([7, 7])
```

PyTorch works the same way:

```
vector = torch.tensor([7, 7])

print(vector.ndim)      # 1                 -- one axis
print(vector.shape)     # torch.Size([2])   -- two elements
print(vector.numel())   # 2                 -- number of elements
print(vector.dtype)     # torch.int64
print(vector.tolist()) # [7, 7]            -- back to a Python list

vector = torch.tensor([7, 7], dtype=torch.float32, device=device)
print(vector.dtype)     # torch.float32
print(vector.device)    # cuda:0            -- on a GPU machine
```

Go 2D: matrices

In NumPy you already know:

```
matrix = np.array([[7, 8], [9, 10]])
```

PyTorch works the same way:

```
matrix = torch.tensor([[7, 8], [9, 10]])

print(matrix.ndim)
print(matrix.shape)
print(matrix.numel())
print(matrix.dtype)
print(matrix.tolist())

matrix = torch.tensor([[7, 8], [9, 10]], dtype=torch.float32, device=device)
print(matrix.dtype)
print(matrix.device)
```

Go 2D: matrices

In NumPy you already know:

```
matrix = np.array([[7, 8], [9, 10]])
```

PyTorch works the same way:

```
matrix = torch.tensor([[7, 8], [9, 10]])

print(matrix.ndim)     # 2                    -- two axes (rows, cols)
print(matrix.shape)    # torch.Size([2, 2])   -- 2 rows, 2 cols
print(matrix.numel())  # 4                    -- number of elements
print(matrix.dtype)    # torch.int64
print(matrix.tolist()) # [[7, 8], [9, 10]]    -- back to a Python list

matrix = torch.tensor([[7, 8], [9, 10]], dtype=torch.float32, device=device)
print(matrix.dtype)    # torch.float32
print(matrix.device)   # cuda:0               -- on a GPU machine
```

Go ND: tensors

In NumPy you already know:

```
tensor = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

PyTorch works the same way:

```
tensor = torch.tensor([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

print(tensor.ndim)
print(tensor.shape)
print(tensor.numel())
print(tensor.dtype)
print(tensor.tolist())

tensor = torch.tensor([[[1, 2], [3, 4]], [[5, 6], [7, 8]]],
                      dtype=torch.float32, device=device)
print(tensor.dtype)
print(tensor.device)
```

Go ND: tensors

In NumPy you already know:

```
tensor = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

PyTorch works the same way:

```
tensor = torch.tensor([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

print(tensor.ndim)     # 3                       -- three axes
print(tensor.shape)    # torch.Size([2, 2, 2])   -- 2 stacked 2x2 blocks
print(tensor.numel())  # 8                       -- number of elements
print(tensor.dtype)    # torch.int64
print(tensor.tolist()) # [[[1, 2], [3, 4]], [[5, 6], [7, 8]]]

tensor = torch.tensor([[[1, 2], [3, 4]], [[5, 6], [7, 8]]],
                      dtype=torch.float32, device=device)
print(tensor.dtype)    # torch.float32
print(tensor.device)   # cuda:0                  -- on a GPU machine
```

Creating tensors

A handful of constructors cover almost everything:

```
z = torch.zeros(size=)
o = torch.ones(size=)
f = torch.full(size=, fill_value=)
n = torch.randn(size=)
u = torch.rand(size=)
r = torch.arange(start=, end=, step=)
l = torch.linspace(start=, end=, steps=)
t = torch.from_numpy(ndarray=)
```

More exotic options available as well:

```
zl = torch.zeros_like(input=z)
ol = torch.ones_like(input=z)
fl = torch.full_like(input=z, fill_value=)
```

Creating tensors

A handful of constructors cover almost everything:

```
z = torch.zeros(size=)                        # all zeros
o = torch.ones(size=)                         # all ones
f = torch.full(size=, fill_value=)            # filled with a constant
n = torch.randn(size=)                        # standard normal
u = torch.rand(size=)                         # uniform in [0, 1)
r = torch.arange(start=, end=, step=)         # evenly spaced range
l = torch.linspace(start=, end=, steps=)      # evenly spaced points
t = torch.from_numpy(ndarray=)                # from a NumPy array
```

More exotic options available as well:

```
zl = torch.zeros_like(input=z)                # zeros shaped like z
ol = torch.ones_like(input=z)                 # ones shaped like z
fl = torch.full_like(input=z, fill_value=)    # constant, shaped like z
```

Indexing & Slicing

Tensor access follows the same rules as NumPy.

```
M = torch.tensor([[ 0,  1,  2,  3],
                  [ 4,  5,  6,  7],
                  [ 8,  9, 10, 11]])

print(M[1])
print(M[1, 3])
print(M[-1])
print(M[:, 2])
print(M[:, -3])
print(M[0:2])
print(M[0:2, 1:3])
print(M[::2])
print(M[M > 5])
```

Indexing & Slicing

Tensor access follows the same rules as NumPy.

```
M = torch.tensor([[ 0,  1,  2,  3],
                  [ 4,  5,  6,  7],
                  [ 8,  9, 10, 11]])

print(M[1])          # tensor([4, 5, 6, 7])
print(M[1, 3])       # tensor(7)
print(M[-1])         # tensor([8, 9, 10, 11])
print(M[:, 2])       # tensor([ 2,  6, 10])
print(M[:, -3])      # tensor([ 1,  5,  9])
print(M[0:2])        # tensor([[0,1,2,3],[4,5,6,7]])
print(M[0:2, 1:3])   # tensor([[1, 2], [5, 6]])
print(M[::2])        # tensor([[0,1,2,3],[8,9,10,11]])
print(M[M > 5])      # tensor([ 6,  7,  8,  9, 10, 11])
```

Arithmetic with Tensors

Arithmetic on tensors is **elementwise**.

```
a = torch.tensor([1, 2, 3])
b = torch.tensor([100, 200, 300])

b + a
b - a
b * a
b / a

b + 10
b - 10
b * 10
b / 10
```

Arithmetic with Tensors

Arithmetic on tensors is **elementwise**.

```
a = torch.tensor([1, 2, 3])
b = torch.tensor([100, 200, 300])

b + a      # tensor([101, 202, 303])      -- position by position
b - a      # tensor([99, 198, 297])
b * a      # tensor([100, 400, 900])      -- Hadamard product
b / a      # tensor([100., 100., 100.])

b + 10     # tensor([110, 210, 310])      -- or against a scalar
b - 10     # tensor([90, 190, 290])
b * 10     # tensor([1000, 2000, 3000])
b / 10     # tensor([10., 20., 30.])
```

Arithmetic with Tensors: Caution!

These operators return a **new tensor**.
The original stays **unchanged** unless you reassign it.

```
b = torch.tensor([100, 200, 300])

b + 10
print(b)

b = b + 10
print(b)

b += 10
print(b)
```

Arithmetic with Tensors: Caution!

These operators return a **new tensor**.
The original stays **unchanged** unless you reassign it.

```
b = torch.tensor([100, 200, 300])

b + 10          # tensor([110, 210, 310])  -- a brand-new tensor
print(b)        # tensor([100, 200, 300])  -- b is untouched!

b = b + 10      # reassign to actually update b
print(b)        # tensor([110, 210, 310])

b += 10         # += works as expected -- updates b too
print(b)        # tensor([120, 220, 320])
```

Arithmetic with Tensors: Broadcasting

The rule

To combine two tensors elementwise, align their shapes **right-to-left**.
Two dimensions are compatible if they are **equal** or **one of them is 1**.

A row $(3,)$ plus a column $(3,1)$ broadcasts to $(3,3)$:

$$\begin{pmatrix} 1 & 2 & 3 \end{pmatrix} + \begin{pmatrix} 10 \\ 20 \\ 30 \end{pmatrix} = \begin{pmatrix} 11 & 12 & 13 \\ 21 & 22 & 23 \\ 31 & 32 & 33 \end{pmatrix}$$

Arithmetic with Tensors: Broadcasting

The rule

To combine two tensors elementwise, align their shapes **right-to-left**.
Two dimensions are compatible if they are **equal** or **one of them is 1**.

A column $(3, 1)$ plus a row $(1, 4)$ broadcasts to $(3, 4)$:

$$\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} + \begin{pmatrix} 10 & 20 & 30 & 40 \end{pmatrix} = \begin{pmatrix} 11 & 21 & 31 & 41 \\ 12 & 22 & 32 & 42 \\ 13 & 23 & 33 & 43 \end{pmatrix}$$

The operation that rules them all

Matrix multiplication is the **cornerstone** that makes the bulk of deep learning work. For example, every linear layer we will encounter is implemented using it.

Matrix multiplication (matmul)

$$C = AB, \quad C_{ij} = \sum_k A_{ik} B_{kj}$$

Shape rule

$(m, k) \times (k, n) \to (m, n)$

The operation that rules them all

```
A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])
B = torch.tensor([[ 1,  2,  3,  4],
                  [ 5,  6,  7,  8],
                  [ 9, 10, 11, 12]])

print(A @ B)
print(torch.matmul(A, B))
```

The operation that rules them all

```
A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])          # shape (2, 3)
B = torch.tensor([[ 1,  2,  3,  4],
                  [ 5,  6,  7,  8],
                  [ 9, 10, 11, 12]])   # shape (3, 4)

print(A @ B)                           # tensor([[ 38,  44,  50,  56],
                                       #         [ 83,  98, 113, 128]])
print(torch.matmul(A, B))              # same result
```

Reshaping using: Transpose

```
A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])

A_T = A.T
print(A_T)
print(A_T.shape)
print(A_T.transpose(0, 1))

T = torch.zeros(2, 3, 4)
T.T
```

Reshaping using: Transpose

```
A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])   # shape (2, 3)

A_T = A.T
print(A_T)                      # tensor([[1, 4], [2, 5], [3, 6]])
print(A_T.shape)                # torch.Size([3, 2])
print(A_T.transpose(0, 1))      # tensor([[1, 2, 3], [4, 5, 6]])

T = torch.zeros(2, 3, 4)        # shape (2, 3, 4)
T.T                             # RuntimeError: .T only supported on 2D tensors
```

Reshaping using: Reshape

Works on any number of dimensions as long as the total elements stays the same.

```
A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])

print(A.reshape(3, 2))

B = torch.zeros(3, 4, 2)
print(B.reshape(6, 4).shape)
print(B.reshape(24).shape)
```

Reshaping using: Reshape

Works on any number of dimensions as long as the total elements stays the same.

```
A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])   # shape (2, 3)

print(A.reshape(3, 2))          # tensor([[1, 2], [3, 4], [5, 6]])

B = torch.zeros(3, 4, 2)        # shape (3, 4, 2) -- 24 elements
print(B.reshape(6, 4).shape)    # torch.Size([6, 4])
print(B.reshape(24).shape)      # torch.Size([24])
```

Reshaping using: View

Works on any number of dimensions as long as the total elements stays the same.

```
A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])

print(A.view(3, 2))

B = torch.zeros(3, 4, 2)
print(B.view(6, 4).shape)
print(B.view(24).shape)
```

Reshaping using: View

Works on any number of dimensions as long as the total elements stays the same.

```
A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])   # shape (2, 3)

print(A.view(3, 2))             # tensor([[1, 2], [3, 4], [5, 6]])

B = torch.zeros(3, 4, 2)        # shape (3, 4, 2) -- 24 elements
print(B.view(6, 4).shape)       # torch.Size([6, 4])
print(B.view(24).shape)         # torch.Size([24])
```

But then, what is the difference?

One always **shares storage** with the original; the other silently **copies** the tensor.

```
A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])

B = A.view(3, 2)
B[0, 0] = 99
print(A)

A_T = A.T
C = A_T.reshape(6)
C[0] = 0
print(A_T)
```

But then, what is the difference?

One always **shares storage** with the original; the other silently **copies** the tensor.

```
A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])

B = A.view(3, 2)
B[0, 0] = 99
print(A)                          # tensor([[99,  2,  3], [4, 5, 6]])

A_T = A.T                         # shape (3, 2)
C = A_T.reshape(6)
C[0] = 0
print(A_T)                        # tensor([[99,  4], [ 2,  5], [ 3,  6]])
```

Reshaping using: Stack & Cat

Stacking two or more tensors of same size to create a new dimension.

```
A = torch.tensor([1, 2, 3])
B = torch.tensor([4, 5, 6])

print(torch.stack([A, B]))
print(torch.stack([A, B], dim=1))
```

We can also concatenate along an **existing** dimension.

```
A = torch.tensor([1, 2, 3])
B = torch.tensor([4, 5, 6])

print(torch.cat([A, B]))
print(torch.cat([A, B], dim=0))
print(torch.cat([A, B], dim=1))
```

Reshaping using: Stack & Cat

Stacking two or more tensors of same size to create a new dimension.

```
A = torch.tensor([1, 2, 3])
B = torch.tensor([4, 5, 6])

print(torch.stack([A, B]))          # tensor([[1, 2, 3], [4, 5, 6]])
print(torch.stack([A, B], dim=1))   # tensor([[1, 4], [2, 5], [3, 6]])
```

We can also concatenate along an **existing** dimension.

```
A = torch.tensor([1, 2, 3])
B = torch.tensor([4, 5, 6])

print(torch.cat([A, B]))            # tensor([1, 2, 3, 4, 5, 6])
print(torch.cat([A, B], dim=0))     # tensor([1, 2, 3, 4, 5, 6])
print(torch.cat([A, B], dim=1))     # IndexError: dim 1 out of range -- A is 1D
```

Reshaping using: Squeeze & Unsqueeze

```
A = torch.zeros(1, 3, 1)

print(A.squeeze().shape)
print(A.squeeze(0).shape)
print(A.squeeze(2).shape)

B = torch.tensor([1, 2, 3])

print(B.unsqueeze(0).shape)
print(B.unsqueeze(1).shape)
```

Reshaping using: Squeeze & Unsqueeze

```
A = torch.zeros(1, 3, 1)            # shape (1, 3, 1)

print(A.squeeze().shape)            # torch.Size([3])
print(A.squeeze(0).shape)           # torch.Size([3, 1])
print(A.squeeze(2).shape)           # torch.Size([1, 3])

B = torch.tensor([1, 2, 3])         # shape (3,)

print(B.unsqueeze(0).shape)         # torch.Size([1, 3])
print(B.unsqueeze(1).shape)         # torch.Size([3, 1])
```

Reshaping using: Permute

The most flexible method when it comes to reshaping.

```
A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])

print(A.permute(1, 0))

T = torch.zeros(2, 3, 4)
print(T.permute(0, 2, 1).shape)
print(T.permute(2, 0, 1).shape)
print(T.permute(2, 1, 0).shape)
```

Similarly to **view**, this method shares memory with the original tensor.

Reshaping using: Permute

The most flexible method when it comes to reshaping.

```
A = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])   # shape (2, 3)

print(A.permute(1, 0))          # tensor([[1, 4], [2, 5], [3, 6]])

T = torch.zeros(2, 3, 4)        # shape (2, 3, 4)
print(T.permute(0, 2, 1).shape) # torch.Size([2, 4, 3])
print(T.permute(2, 0, 1).shape) # torch.Size([4, 2, 3])
print(T.permute(2, 1, 0).shape) # torch.Size([4, 3, 2])
```

Similarly to **view**, this method shares memory with the original tensor.

Tensor Aggregation: Reductions

Reductions collapse a tensor along a `dim`:

```
A = torch.tensor([[1., 2., 3.],
                  [4., 5., 6.]])

print(A.sum(dim=0))
print(A.sum(dim=1))
print(A.sum(dim=1, keepdim=True))

print(A.mean(dim=1))
print(A.max(dim=1))
print(A.min(dim=0))

print(A.argmax(dim=1))
print(A.argmin(dim=0))
```

Tensor Aggregation: Reductions

Reductions collapse a tensor along a `dim`:

```
A = torch.tensor([[1., 2., 3.],
                  [4., 5., 6.]])  # shape (2, 3)

print(A.sum(dim=0))               # tensor([5., 7., 9.])
print(A.sum(dim=1))               # tensor([ 6., 15.])          -- shape (2,)
print(A.sum(dim=1, keepdim=True)) # tensor([[ 6.], [15.]])      -- shape (2, 1)

print(A.mean(dim=1))              # tensor([2., 5.])
print(A.max(dim=1))               # values: tensor([3., 6.])
print(A.min(dim=0))               # values: tensor([1., 2., 3.])

print(A.argmax(dim=1))            # tensor([2, 2])
print(A.argmin(dim=0))            # tensor([0, 0, 0])
```

Datasets & DataLoaders

getting data into the model, batch by batch

Tensors & Data

Tensors job is to **represent data in a numerical way**.

Example: an image as tensor

A colour image is a tensor of shape `[3, 224, 224]`.

Images, text, tables, audio is represented as a **tensor**.

To train a model we feed it with **many** such tensors.

This is why we need **Datasets & DataLoaders**.

Datasets & DataLoaders

Models train on **mini-batches**, not one giant array. PyTorch splits that job in two:

Dataset

Knows how to fetch **one sample** (and its label) by index.
You implement `__len__` and `__getitem__`.

DataLoader

Wraps a Dataset and serves **shuffled mini-batches**.
Provides us with automatic batching and optional **parallel workers**.

A custom Dataset, served by a DataLoader

```
from torch.utils.data import Dataset, DataLoader

class MyDataset(Dataset):
    def __init__(self, X, y):
        self.X, self.y = X, y
    def __len__(self):
        return len(self.X)
    def __getitem__(self, i):
        return self.X[i], self.y[i]

loader = DataLoader(MyDataset(X, y), batch_size=32, shuffle=True)

for xb, yb in loader:
    ...
```

Plenty other options are also available (e.g. `torchvision.datasets`).

Models' representation power

from a single hyperplane to any continuous function

Why neural networks?

From a straight line to a curved boundary

A **linear** model separates classes with a single **straight** boundary. Many real problems are not linear: their structure cannot be captured by any single hyperplane.

XOR

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

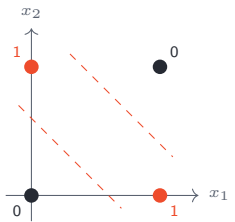
Why neural networks?

From a straight line to a curved boundary

A **linear** model separates classes with a single **straight** boundary. Many real problems are not linear: their structure cannot be captured by any single hyperplane.

XOR

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



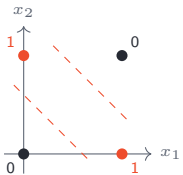
two lines needed, not one

What does the trick?

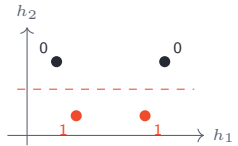
A single hidden layer!

$$\mathbf{h} = \phi(W_1\mathbf{x} + \mathbf{b}_1), \qquad \hat{y} = \sigma(W_2\mathbf{h} + b_2)$$

input space



hidden space



There is still hope!

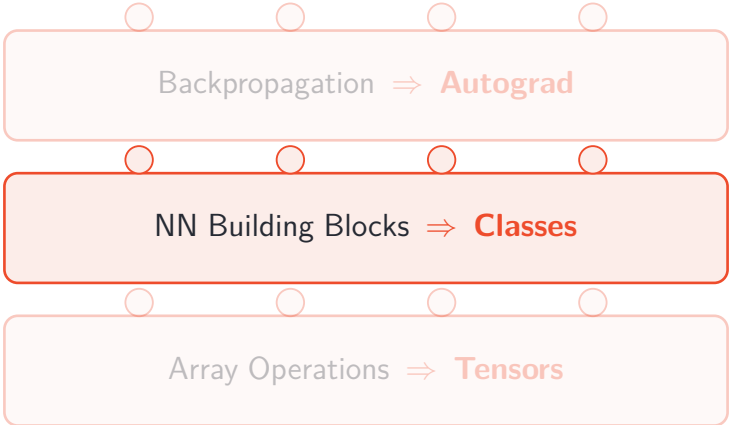
The Universal Approximation Theorem

For any continuous, non-polynomial ϕ, any continuous $f : [0,1]^d \to \mathbb{R}$ and any $\varepsilon > 0$, there exist a width H, weights $\mathbf{w}_i$, v_i and biases b_i s.t.

$$\left| f(\mathbf{x}) - \sum_{i=1}^{H} v_i \, \phi(\mathbf{w}_i^\top \mathbf{x} + b_i) \right| < \varepsilon \quad \forall \mathbf{x} \in [0,1]^d.$$

In plain words: **a single hidden layer of sufficient width can approximate any continuous function arbitrarily well.**

PyTorch Lego Pieces



NN Building Blocks

the classes that make everything click

Layers and the forward pass

A network is a stack of **layers**. The basic one is the **linear** layer,

$$y = Wx + b, \qquad W \in \mathbb{R}^{\text{out} \times \text{in}}, \quad b \in \mathbb{R}^{\text{out}}.$$

Its entries W, b are the **learnable parameters**.

In a framework

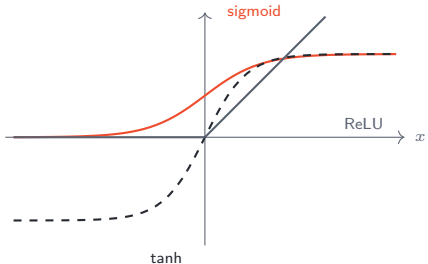
We define a network as an object that owns its layers (so their parameters are tracked automatically) and defines a `forward` method describing how an input flows through it. The layers are just tensor operations; nothing magic happens.

Linear layers is not enough

Composing linear maps gives **another linear map**:

$$x \mapsto W_2(W_1 x + b_1) + b_2 \;=\; Wx + b.$$

To represent **non-linear** functions we insert a non-linear **activation** ϕ between layers.



Sigmoid $\sigma(x) = \dfrac{1}{1 + e^{-x}}$

Tanh $\tanh(x)$

ReLU $\max(0,\, x)$

The core class: nn.Module

Every PyTorch model is a **class** that subclasses `nn.Module`.
You inherit all of its machinery and fill in two methods:

```
class MLP(nn.Module):

    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(2, 4)
        self.fc2 = nn.Linear(4, 1)

    def forward(self, x):
        h = self.fc1(x)
        h = torch.tanh(h)
        return self.fc2(h)
```

Anything assigned as `self.` is registered in `model.parameters()`.

The building-block catalogue

Architectures are assembled from ready-made classes:

Kind	Examples
Layers	`nn.Linear`, `nn.Conv2d`, `nn.Embedding`
Activations	`nn.ReLU`, `nn.Sigmoid`, `nn.Tanh`
Normalisation	`nn.BatchNorm2d`, `nn.LayerNorm`
Containers	`nn.Sequential`, `nn.ModuleList`
Losses	`nn.BCEWithLogitsLoss`, `nn.MSELoss`
Optimizers	`torch.optim.SGD`, `torch.optim.AdamW`

Building blocks in action

```
import torch.nn as nn

class SmallNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv    = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.bn      = nn.BatchNorm2d(16)
        self.relu    = nn.ReLU()
        self.flatten = nn.Flatten()
        self.fc      = nn.Linear(16 * 28 * 28, 10)

    def forward(self, x):
        x = self.conv(x)
        x = self.bn(x)
        x = self.relu(x)
        x = self.flatten(x)
        return self.fc(x)
```

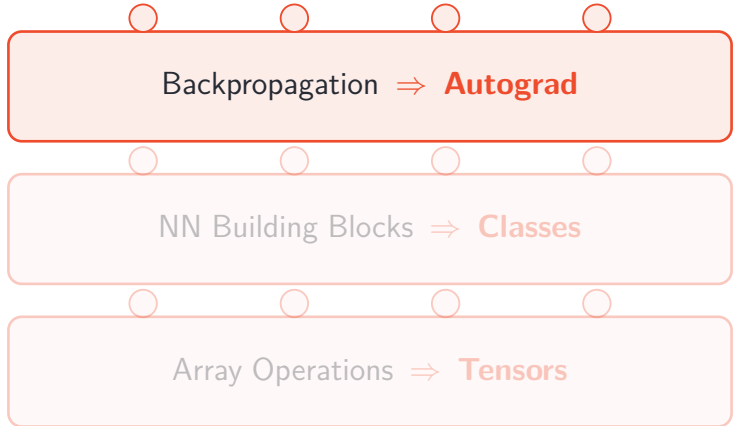
Building blocks in action: Sequential `__init__`

```
class SmallNetSeq(nn.Module):
    def __init__(self, in_channels=1, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(in_channels, 16, kernel_size=3, padding=1),
            nn.BatchNorm2d(16),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16 * 14 * 14, 128),
            nn.ReLU(),
            nn.Linear(128, num_classes),
        )
```

Building blocks in action: Sequential `forward`

```
    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)
```

PyTorch Lego Pieces



Backpropagation

autograd, the driving horse of modern AI

Why not just use NumPy Arrays?

A tensor is a NumPy array with **two extra powers**:

1. **It can live on a GPU.** Move it to the device and every operation runs there, in parallel over thousands of cores.
2. **It tracks the operations applied to it**, so the framework can compute gradients automatically.

The second superpower: Tracking gradients

Set `requires_grad=True` and a tensor begins **recording every operation** applied to it. Ask for a gradient and PyTorch replays that record backwards:

```
x = torch.tensor(2.0, requires_grad=True)
y = x**3
y.backward()
x.grad            # tensor(12.)  ==  3 * x**2  at x = 2
```

A soulless architecture

We can now *build* a network, a **randomly initialised** one. Its weights are noise and its predictions are meaningless: the structure is there, yet nothing has been *learned*.

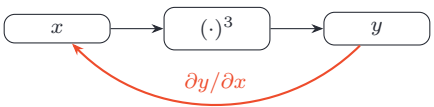
The missing ingredient

To make the network **learn from data** we need to measure how wrong it is and nudge every weight the right way.

Everything flowing through the network is a tensor giving us gradients for "free".

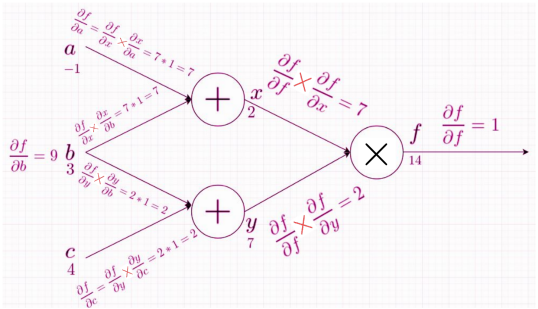
But, how does this work?

Mark a tensor as requiring gradients, do any differentiable computation, then ask for the gradient of the result. The framework records every operation in a **computation graph** and walks it *in reverse*, applying the chain rule.



Backpropagation: what is it *really*?

A linear-time dynamic program for computing derivatives



A miracle of dynamic program

Backprop propagates the gradients along the graph, **reusing** shared sub-results.

Functions are computation graphs

A **composite function** is an ordered series of operations, each one a function only of the preceding results. Introduce an intermediate variable per operation, then draw the graph:

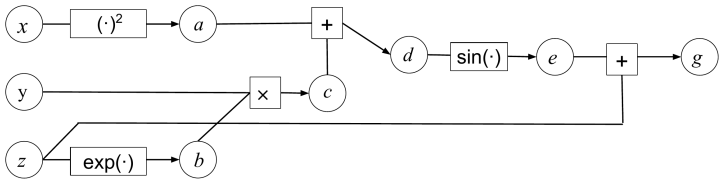
Example:

$$f(x, y, z) = z + \sin(x^2 + y \times \exp(z))$$

$a = x^2$ $\quad d = a + c$

$b = \exp(z)$ $\quad e = \sin(d)$

$c = y \times b$ $\quad g = e + z$

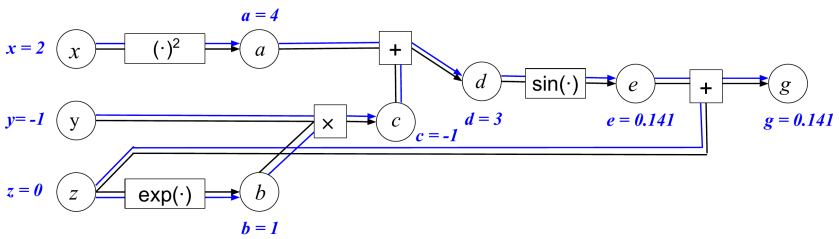


Forward pass: compute (and store) the values

Set the inputs, then evaluate each node from its parents, left to right. This computes the output *and* stores every intermediate value for the backward pass.

Example:

$$f(x, y, z) = z + \sin(x^2 + y \times \exp(z))$$



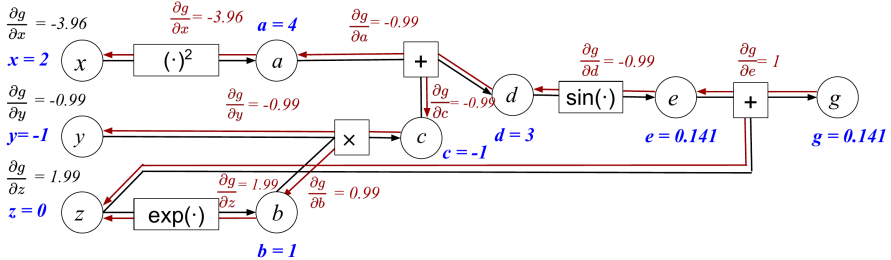
$$f(2, -1, 0) = g = 0.141$$

Backward pass: propagate the gradients

Start at the output and walk *in reverse*. Each edge multiplies by its local impact.

Example:

$$f(x, y, z) = z + \sin(x^2 + y \times \exp(z))$$



$$\nabla f(2, -1, 0) = \langle \frac{\partial g}{\partial x}, \frac{\partial g}{\partial y}, \frac{\partial g}{\partial z} \rangle = \langle -3.96, -0.99, 1.99 \rangle$$

What this means every time you call `.backward()`

The whole loop, in one mechanism

During the **forward pass**, PyTorch builds the computation graph and caches intermediate values. `loss.backward()` then runs **exactly this reverse sweep**. One pass, every gradient.

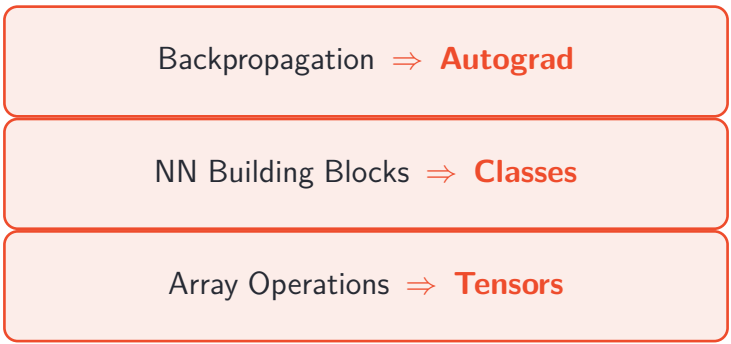
Backpropagation reduces gradient computation to **a single reverse pass**.

The same cost as evaluating the function!!!

That efficiency is what makes training billion-parameter models possible at all.

It is, possibly, **the single most important algorithm in history**.

PyTorch Lego Pieces



How a network learns

loss, gradient descent, and the training loop

Loss functions: turning “wrong” into a number

Training needs a single scalar measuring how wrong the predictions are; we then minimise it.

Classification with cross-entropy

For target $y \in \{0, 1\}$ and predicted probability $\hat{p} = \sigma(z)$ from logit z:

$$\mathcal{L}_{\text{BCE}} = -\big[\, y \log \hat{p} + (1 - y) \log(1 - \hat{p}) \,\big].$$

Regression with mean squared error

For continuous targets:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2.$$

The building-block catalogue

Architectures are assembled from ready-made classes:

Kind	Examples
Layers	`nn.Linear`, `nn.Conv2d`, `nn.Embedding`
Activations	`nn.ReLU`, `nn.Sigmoid`, `nn.Tanh`
Normalisation	`nn.BatchNorm2d`, `nn.LayerNorm`
Containers	`nn.Sequential`, `nn.ModuleList`
Losses	`nn.BCEWithLogitsLoss`, `nn.MSELoss`
Optimizers	`torch.optim.SGD`, `torch.optim.AdamW`

Gradient descent

The loss $\mathcal{L}(\theta)$ is a function of all parameters θ. Its gradient $\nabla_\theta \mathcal{L}$ points in the direction of steepest *increase*, so we step the opposite way:

$$\theta_{t+1} = \theta_t - \eta \, \nabla_\theta \mathcal{L}(\theta_t),$$

where η is the **learning rate**.

Optimizers: from SGD to Adam

The gradient says *which way* is downhill.
The **optimizer** decides *how far* to step, and owns the update rule.

SGD	Adam
$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$, global η.	Per-parameter adaptive step sizes from running averages of the gradient.

The building-block catalogue

Architectures are assembled from ready-made classes:

Kind	Examples
Layers	`nn.Linear`, `nn.Conv2d`, `nn.Embedding`
Activations	`nn.ReLU`, `nn.Sigmoid`, `nn.Tanh`
Normalisation	`nn.BatchNorm2d`, `nn.LayerNorm`
Containers	`nn.Sequential`, `nn.ModuleList`
Losses	`nn.BCEWithLogitsLoss`, `nn.MSELoss`
Optimizers	`torch.optim.SGD`, `torch.optim.AdamW`

The training loop

Whatever the model or data, one optimisation step is always these stages, in this order:

1. **Zero the gradients**: they accumulate by default, so clear last step's.
2. **Forward pass**: run the inputs through the model to get predictions.
3. **Compute the loss**: compare predictions against targets.
4. **Backward pass**: autograd fills every parameter's gradient.
5. **Step**: the optimizer nudges each parameter downhill.

The training loop: PyTorch

```
for epoch in range(num_epochs):
    for X_batch, y_batch in dataloader:

        # 1. zero the gradients
        optimizer.zero_grad()

        # 2. forward pass
        y_pred = model(X_batch)

        # 3. compute the loss
        loss = criterion(y_pred, y_batch)

        # 4. backward pass
        loss.backward()

        # 5. step
        optimizer.step()
```

Using the model

```
model.eval()

with torch.no_grad():
    for X_batch, y_batch in test_loader:
        y_pred = model(X_batch)
        loss = criterion(y_pred, y_batch)
```

Scaling up: mini-batch gradient descent

On a handful of points we can compute the gradient over the **whole** dataset each step (full-batch). With thousands or millions of rows that is wasteful, so each update instead uses a small random **mini-batch**.

Why this is sound

The true gradient is an average over all N samples. A batch of size B computes the same average over a random subset,

$$\hat{g}_B = \frac{1}{B} \sum_{i \in \text{batch}} \nabla \ell_i,$$

an **unbiased estimator** of the full gradient. We trade one exact step for **many cheap, slightly noisy** steps — usually a far better deal, and the noise can even help escape poor minima.

Summary — the three primitives

Tensors

NumPy arrays with a GPU and autograd: `shape` / `ndim` / `numel` / `dtype`, broadcasting, `dim`-wise reductions, matrix vs. elementwise products.

The forward pass

Layers (linear maps) composed with **non-linear activations**, the one ingredient that makes depth meaningful and lets a network bend its decision boundary.

The backward pass

A loss turns error into a number; autograd's reverse walk yields every gradient; gradient descent steps the parameters downhill.

Lecture Roadmap

1. **Tensors**: what they are, and the operations every model is built from.
2. **Neural Networks**: `nn.Module`, linear layers, and why non-linear activations are essential.
3. **Expressive power**: linear separability, XOR, and the universal approximation theorem.
4. **Autograd & backpropagation**: how a tensor remembers its own computation, and how reverse-mode AD turns it into gradients efficiently.
5. **Training**: gradient descent, loss functions, the canonical loop, and mini-batches.

Now, to the keyboard

Time to build and train these networks yourself.